

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ АРХИТЕКТУРНЫХ РЕШЕНИЙ.....	8
1.1. Актуальность задачи интеллектуального управления медиаконтентом	8
1.2. Обзор существующих решений.....	9
1.2.1. Яндекс.Диск.....	9
1.2.2. Облако Mail.ru	10
1.2.3. СбeрДиск.....	11
1.2.4. Google Photos	12
1.2.5. Сравнительный анализ	13
1.3. Современные подходы к автоматическому тегированию изображений	15
1.3.1. Мультимодальные модели контрастного обучения	15
1.4. Семантический поиск на основе векторных представлений	17
1.4.1. Векторные базы данных	17
1.4.2. Гибридный поиск	18
1.5. Распознавание лиц и кластеризация персон.....	19
1.5.1. Архитектура подсистемы распознавания лиц.....	19
1.5.2. Задача кластеризации лиц	20
1.5.3. Инкрементальная кластеризация и UX-аспекты.....	20
1.6. Обоснование архитектуры системы и выбора технологий.....	21
1.6.1. Двухуровневое представление контента	21

1.6.2. Архитектура программных компонентов	22
1.6.3. Обоснование выбора языка программирования и фреймворка	23
1.7. Постановка задач разработки.....	24
Выводы к главе 1	26
2.1. Структура программной системы.....	28
2.1.1. Архитектура и компоненты	28
2.1.2. Поток данных при обработке файла	29
2.1.3. Структура программного кода.....	31
2.2. Реализация серверной части	32
2.2.1. Структура базы данных	32
2.2.2. Программный интерфейс REST API	33
2.2.3. Аутентификация пользователей	34
2.2.4. Загрузка и выдача файлов	35
2.3. Реализация ML-конвейера.....	36
2.3.1. Извлечение метаданных и геокодирование.....	36
2.3.2. Семантическое индексирование на основе CLIP.....	38
2.3.3. Zero-shot классификация типа изображения	39
2.3.4. Распознавание и кластеризация лиц	40
2.3.4.1. Алгоритм инкрементальной кластеризации.....	42
2.3.5. Обработка видеоматериалов	44
2.4. Реализация гибридного поиска.....	45
2.4.1. Параметры запроса.....	45
2.4.2. Алгоритм обработки запроса	45
2.4.3. Примеры запросов.....	47

2.5. Пользовательский интерфейс	47
2.5.1. Структура интерфейса	48
2.5.2. Главная страница.....	48
2.5.3. Загрузка файлов и индикация статуса	49
2.5.4. Раздел «Люди» и редактирование персон	50
2.6. Развёртывание системы	51
2.7. Экспериментальное исследование	52
2.7.1. Тестовые выборки	52
2.7.2. Качество zero-shot классификации	52
2.7.3. Качество семантического поиска	53
2.7.4. Качество кластеризации лиц	55
2.7.5. Эксплуатационные характеристики	56
Выводы к главе 2	57

ВВЕДЕНИЕ

Стремительный рост объёмов производимого и потребляемого цифрового медиаконтента стал одной из определяющих характеристик современной информационной среды. По данным аналитических исследований, ежедневно в мире создаётся свыше нескольких миллиардов фотографий и видеозаписей, при этом значительная их часть хранится в личных архивах пользователей и предприятий. Подобный рост сопровождается принципиальным изменением способов хранения и взаимодействия пользователей со своими медиа архивами. Пользователи все чаще переходят на онлайн хранилища для обеспечения безопасности и доступности своих коллекций.

Существующие популярные облачные сервисы — Яндекс.Диск, Облако Mail.ru, Google Photos и другие — демонстрируют принципиальную возможность построения интеллектуального медиа-хранилища, которое позволит пользователю удобно осуществлять навигацию по своей коллекции. Качество так системы обработки напрямую зависит от используемых нейросетевых моделей и от того, насколько грамотно они интегрированы между собой в единый конвейер обработки данных.

Современный уровень развития методов машинного обучения и компьютерного зрения создаёт богатый инструментарий для решения подобных задач. Однако интеграция нескольких разнородных моделей в единую систему с приемлемыми пользовательскими характеристиками по-прежнему остаётся нетривиальной инженерной задачей.

Актуальность работы обусловлена стремительным ростом объёмов цифрового медиаконтента и необходимостью разработки автоматизированных систем его организации и поиска. Интеграция нескольких нейросетевых моделей в единую программную систему представляет собой нетривиальную инженерную задачу, требующую

комплексного подхода к проектированию архитектуры и подбору ML-компонентов.

Цель работы — разработка облачного хранилища медиафайлов с автоматическим тегированием, распознаванием лиц и семантическим поиском на основе интеграции нейросетевых моделей.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Проанализировать предметную область интеллектуального управления медиаконтентом и рассмотреть существующие облачные сервисы, выявить их функциональные особенности.
2. Исследовать современные методы автоматического тегирования изображений, семантического поиска по содержанию, распознавания и кластеризации лиц; обосновать выбор моделей и алгоритмов.
3. Спроектировать архитектуру программной системы, обеспечивающую модульность и расширяемость.
4. Разработать программный комплекс, включающий серверную часть, систему хранения и веб-интерфейс пользователя.
5. Разработать систему автоматической разметки медиаконтента и комбинированного поиска, сочетающего фильтрацию по структурированным метаданным с семантическим поиском на основе векторных представлений.
6. Провести экспериментальную оценку качества ML-компонентов и системы в целом на тестовых выборках, а также обосновать выбор ключевых параметров настройки моделей.

Объект исследования — процесс организации, хранения и поиска медиафайлов в цифровых архивах пользователей.

Предмет исследования — методы и программные средства автоматического тегирования и семантического поиска медиафайлов на основе нейросетевых моделей.

Методологическую основу исследования составили: системный подход к проектированию программных решений; современные методы машинного обучения и компьютерного зрения; принципы микросервисной и модульной архитектуры; принципы итеративной разработки программного обеспечения.

Практическая значимость работы определяется тем, что разработанные архитектурные решения и подходы к интеграции нейросетевых моделей могут быть применены разработчиками при создании систем интеллектуальной обработки медиаконтента — как в составе корпоративных решений для управления цифровыми активами, так и в потребительских облачных сервисах.

Научная новизна заключается в обосновании и практической реализации двухуровневой модели представления медиаконтента, объединяющей дискретный навигационный слой (теги, кластеры, метаданные) и непрерывный векторный слой (мультимодальные эмбединги), а также в разработке гибридной схемы поиска, совместно использующей оба представления.

Структура работы. Дипломная работа состоит из введения, двух глав, заключения, списка литературы и приложений. В первой главе проводится анализ предметной области, обзор существующих сервисов и современных подходов к автоматическому тегированию и семантическому поиску, обосновывается выбор технологического стека и архитектуры. Вторая глава посвящена разработке программного продукта: проектированию структурных и функциональных моделей, реализации ML-конвейера, бэкенда и пользовательского интерфейса, а также экспериментальной оценке. В приложениях приведены техническое задание, диаграммы UML и фрагменты исходного кода. приложениях приведены техническое задание, диаграммы UML и фрагменты исходного кода.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ АРХИТЕКТУРНЫХ РЕШЕНИЙ

1.1. Актуальность задачи интеллектуального управления медиаконтентом

Объёмы создаваемого пользовательского медиаконтента увеличиваются нелинейно на протяжении последних двух десятилетий. По оценкам отраслевых аналитических агентств, к 2025 году в мире ежедневно создаётся порядка 5,3 миллиарда фотографий, а суммарный объём пользовательских медиаархивов исчисляется зеттабайтами [1]. Распространение мобильных устройств со встроенными камерами высокого разрешения и удешевление облачного хранения снимают практические ограничения на объём личных архивов.

Параллельно изменяется характер взаимодействия пользователей со своими архивами. Традиционная иерархия каталогов с осмысленно именованными директориями с ростом объёмов архивов утратила эффективность: ручная классификация тысяч единиц контента становится непосильной для среднего пользователя.

В качестве альтернативы сформировалась модель, в которой основным способом доступа к содержимому архива становятся поиск и фильтрация по автоматически извлекаемым атрибутам контента: техническим метаданным (дата, геолокация, тип файла), результатам автоматического распознавания (объекты, сцены, люди), а также скрытым векторным представлениям, обеспечивающим поиск по семантическому сходству.

Соответствующий класс программных продуктов представлен сервисами Яндекс.Диск, Облако Mail.ru, VK Облако, СберДиск, Google Photos. Качество интеллектуальной обработки в них определяется применяемыми нейросетевыми моделями: чем современнее и точнее модели,

тем более релевантные результаты получает пользователь. Прогресс в области мультимодальных моделей (CLIP), распознавания лиц (ArcFace), векторного поиска (HNSW, Qdrant) за последние годы существенно расширил круг задач, которые могут быть решены автоматически и с высоким качеством.

Таким образом, разработка программного решения для интеллектуального управления медиаконтентом, обеспечивающего автоматическое тегирование, распознавание лиц и семантический поиск на основе интеграции нейросетевых моделей, представляет собой актуальную задачу, обладающую как теоретической, так и практической значимостью.

1.2. Обзор существующих решений

В сети Интернет сегодня доступно значительное число облачных сервисов, обеспечивающих хранение и интеллектуальную организацию фото- и видеоконтента. Под интеллектуальным медиахранилищем здесь понимается сервис, который не ограничивается приёмом, хранением и выдачей файлов, а извлекает из загружаемого контента содержательные признаки (объекты, лица, места) и предоставляет пользователю развитые средства поиска и навигации.

Для подробного анализа отобраны четыре российских сервиса (Яндекс.Диск, Облако Mail.ru, VK Облако, СберДиск) и один зарубежный аналог (Google Photos), рассматриваемый как эталонное решение, задающее верхнюю планку функциональности.

1.2.1. Яндекс.Диск

Яндекс.Диск — облачное хранилище общего назначения от компании «Яндекс» с встроенным разделом «Фото»; аудитория сервиса на 2024 год превышает 50 миллионов человек. Сервис обеспечивает автоматическую загрузку медиафайлов с мобильных устройств, распознавание объектов и

сцен на фотографиях, распознавание лиц с группировкой по людям, группировку по событиям и местам, а также семантический поиск по фотографиям с использованием естественно-языковых запросов на русском языке.

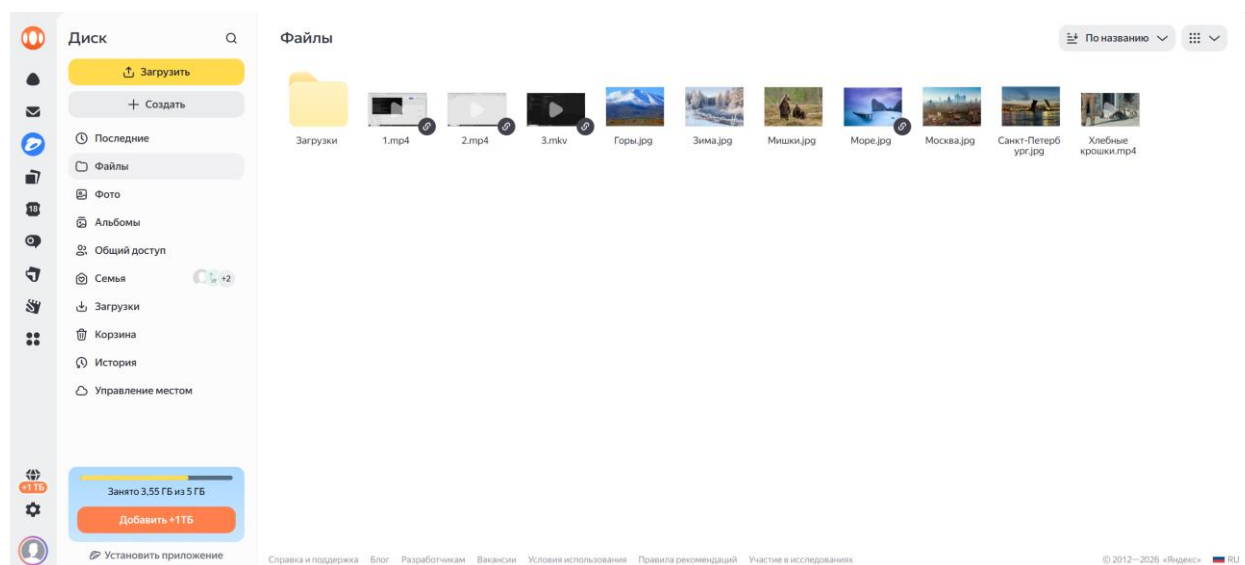


Рисунок 1.1. Главная страница раздела «Фото» сервиса Яндекс.Диск

Интерфейс (рисунок 1.1) выстроен как хронологическая лента с группировкой по датам и событиям; на боковой панели — навигация по людям, местам и альбомам. Сильные стороны решения — высокое качество распознавания на русскоязычном контенте и развитый русскоязычный поиск.

1.2.2. Облако Mail.ru

Облако Mail.ru — облачное хранилище данных от компании VK, на 2024 год обслуживающее свыше 30 миллионов пользователей. В составе сервиса предусмотрен раздел «Фото» с автоматической загрузкой контента, распознаванием лиц с группировкой по персонам, распознаванием объектов и сцен (с более узким набором категорий, чем у конкурентов), группировкой по геолокации, автоматическими ретроспективными подборками.

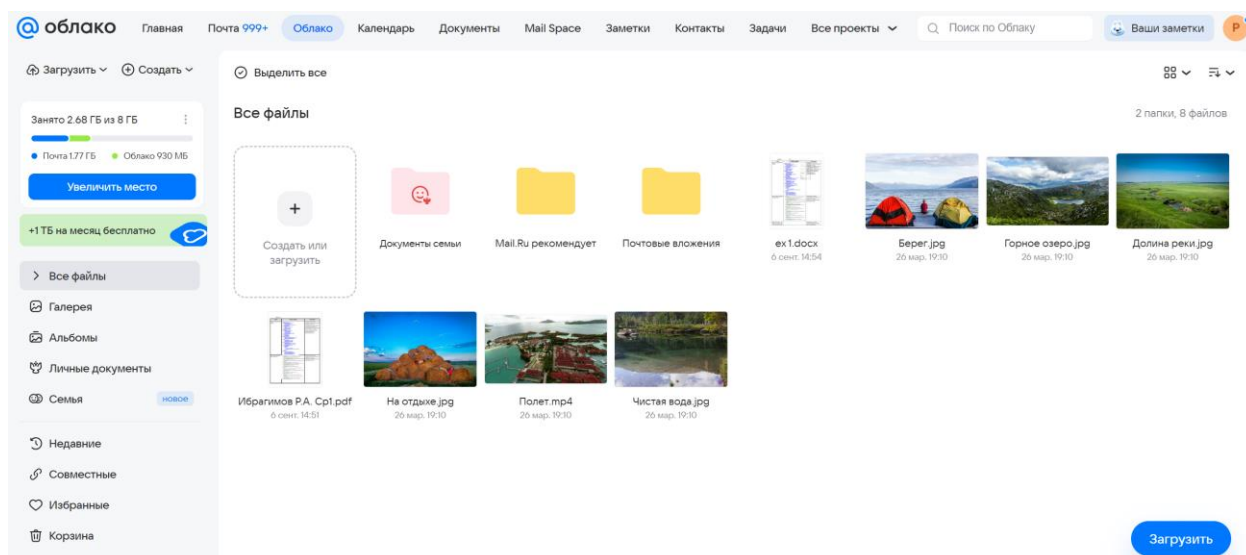


Рисунок 1.2. Главная страница сервиса Облако Mail.ru

По возможностям интеллектуальной обработки (рисунок 1.2) сервис уступает Яндекс.Диску — прежде всего в семантическом поиске, который здесь ограничен поиском по распознанным меткам и не понимает свободно сформулированные запросы.

По сравнению с Яндекс.Диском и Облаком Mail.ru сервис (рисунок 1.2) меньше внимания уделяет развитому семантическому поиску; акцент смещён к интеграции с продуктами экосистемы VK.

1.2.3. СберДиск

СберДиск — облачное хранилище от ПАО «Сбербанк России», входящее в экосистему «Сбер»; аудитория превышает 15 миллионов клиентов. В части работы с медиаконтентом сервис обеспечивает автоматическую загрузку фото и видео, хронологическую и геолокационную группировку, ручное создание альбомов.

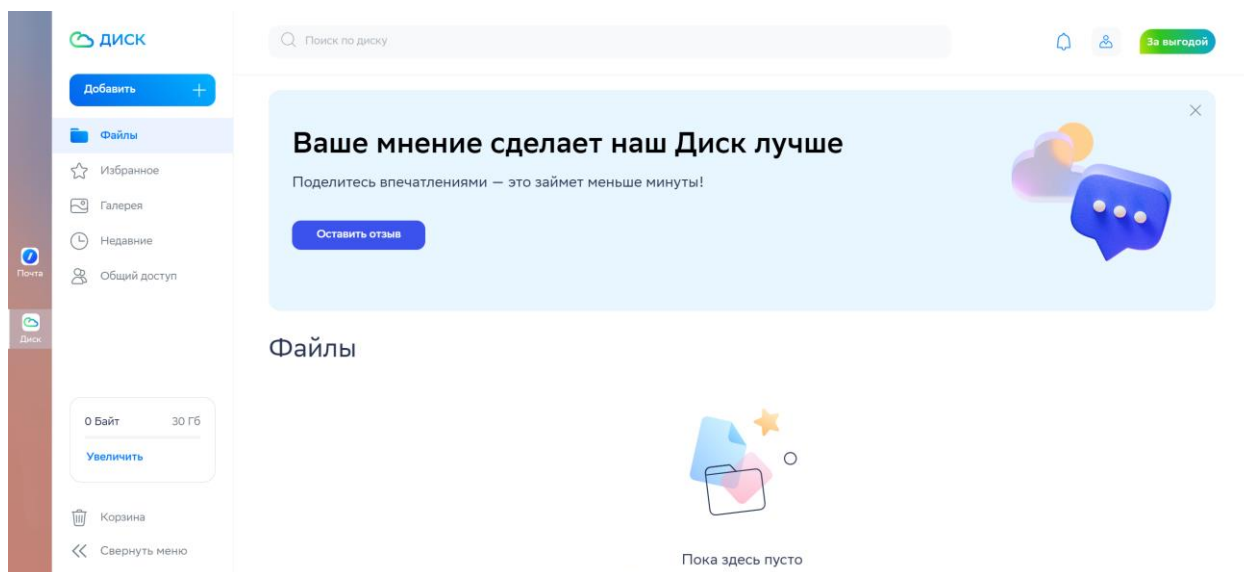


Рисунок 1.3. Главная страница сервиса СберДиск

На момент исследования (рисунок 1.3) СберДиск заметно отстаёт от Яндекс.Диска и Облака Mail.ru в интеллектуальной обработке медиа: автоматическое распознавание лиц и семантический поиск по содержимому фотографий не реализованы либо находятся в стадии разработки.

1.2.4. Google Photos

Google Photos — наиболее зрелое решение своего класса с аудиторией свыше одного миллиарда пользователей. В работе рассматривается как эталонный зарубежный аналог, задающий верхнюю планку функциональности. Сервис обеспечивает распознавание объектов и сцен с поддержкой тысяч категорий, высокоточное распознавание лиц, распознавание текста на фотографиях (OCR), семантический поиск по естественно-языковым запросам, понимающий смысл запроса (по запросу «закат на пляже» возвращаются релевантные фотографии без явной разметки), автоматические подборки и видеомонтажи на основе распознанных событий.

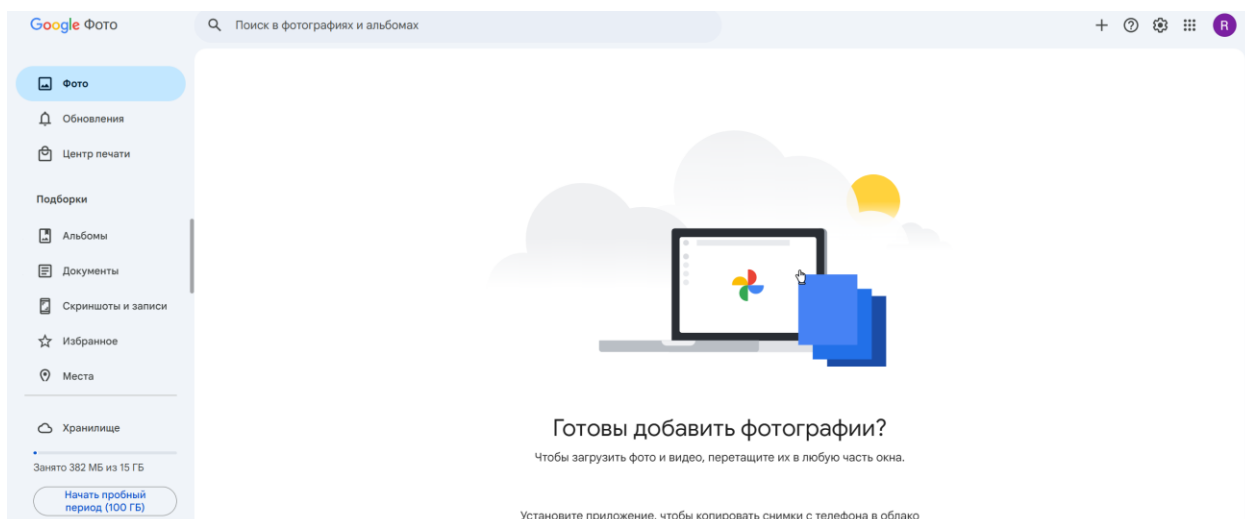


Рисунок 1.4. Главная страница сервиса Google Photos

Технологическую основу сервиса (рисунок 1.4) составляют современные модели машинного обучения, разрабатываемые подразделениями Google Research и Google DeepMind.

1.2.5. Сравнительный анализ

Сводные функциональные характеристики рассмотренных сервисов представлены в таблице 1.1.

Характеристика	Яндекс.Диск	Облако Mail.ru	СберДиск	Google Photos
Объём пользователей	более 50 млн	более 30 млн	более 15 млн	более 1 млрд
Распознавание лиц	+	+	—	+
Распознавание объектов и сцен	+	+	—	+
Семантический поиск по тексту	+	частично	—	+
Группировка по геолокации	+	+	+	+

Характеристика	Яндекс.Диск	Облако Mail.ru	СберДиск	Google Photos
Группировка по событиям/датам	+	+	+	+
Бесплатный объём (ГБ)	5	8	30	15

Таблица 1.1. Сравнительный анализ сервисов интеллектуального хранения медиаконтента

Наиболее зрелым среди отечественных сервисов выглядит Яндекс.Диск: распознавание лиц, объектов, сцен, семантический поиск по русскоязычным запросам работают на приемлемом уровне. Облако Mail.ru и VK Облако реализуют близкий набор функций, но с заметными ограничениями, особенно в семантическом поиске. СберДиск по интеллектуальной обработке медиа сегодня заметно отстаёт от конкурентов. Флагманским зарубежным облачным сервисом является Google Photo

На основании выполненного обзора сформулированы функциональные требования к разрабатываемой системе:

- распознавание объектов, сцен и категорий контента (фото, скриншот, документ, чек);
- распознавание лиц с автоматической кластеризацией по персонам
- геолокационная группировка с поддержкой обратного геокодирования;
- хронологическая организация контента с группировкой по периодам;
- семантический поиск
- веб-интерфейс с возможностью просмотра, навигации и поиска по архиву.

1.3. Современные подходы к автоматическому тегированию изображений

Под автоматическим тегированием изображений понимается процесс присвоения изображению набора дискретных меток, отражающих его содержимое и предназначенных для последующего использования в задачах фильтрации, навигации и поиска. Эволюция подходов к решению этой задачи отражает общее развитие методов компьютерного зрения [5].

Классификация по фиксированному набору категорий — классический подход, состоящий в обучении модели многоклассовой классификации на заранее размеченных наборах данных (ImageNet). Современные модели (ResNet [6], EfficientNet, ConvNeXt) достигают точности классификации, превышающей 85 % top-1 accuracy. Преимуществами подхода являются вычислительная эффективность и интерпретируемость; недостатками — жёсткая фиксация набора возможных меток и смещённое покрытие пространства содержаний.

Детекция объектов и сегментация — модели семейства YOLO [7], DETR, Mask R-CNN обнаруживают на изображении отдельные объекты с указанием их пространственных координат. Применение детекторов для тегирования даёт более детальные метки, но для бытовых медиаархивов число генерируемых меток оказывается избыточным, а сами теги получают низкую различительную способность.

1.3.1. Мультимодальные модели контрастного обучения

Принципиальный сдвиг в подходах к задачам понимания изображений произошёл с появлением мультимодальных моделей, обученных методом контрастного обучения на парах «изображение — текстовое описание». Наиболее известной моделью данного класса является CLIP (Contrastive Language–Image Pre-training), представленная OpenAI в 2021 году [8]. Модель

обучена на 400 миллионах пар «изображение — подпись» и состоит из двух параллельных энкодеров: визуального (Vision Transformer или модифицированный ResNet) и текстового (трансформер). Обучение производится так, чтобы для согласованной пары косинусное сходство выходных векторов было максимальным, а для несогласованной — минимальным.

Фундаментальное значение модели CLIP для рассматриваемой предметной области определяется следующими свойствами:

1. Универсальность семантического представления. Изображения и тексты отображаются в единое векторное пространство, в котором близость векторов соответствует семантической близости содержимого. Это позволяет осуществлять поиск изображений по текстовым описаниям без какой-либо разметки.
2. Способность к классификации в режиме zero-shot. Для классификации изображения по произвольному набору категорий достаточно сформулировать категории в виде текстовых описаний, вычислить их векторные представления и сравнить с вектором изображения. Дополнительное обучение модели не требуется.
3. Высокое качество на широком диапазоне предметных областей, превосходящее во многих задачах модели, специально дообученные на соответствующих доменах.

В задаче автоматического тегирования модель CLIP применяется двумя способами. Первый — построение векторного представления изображения и сохранение его в векторной базе данных для последующего семантического поиска. Второй — применение CLIP в режиме zero-shot классификации с заранее заданным набором категорий, выходом которой является ограниченный набор дискретных тегов. В настоящей работе оба способа используются совместно, что обосновано в разделе 1.6.

1.4. Семантический поиск на основе векторных представлений

Семантический поиск извлекает из коллекции контента элементы, релевантные запросу пользователя по смыслу, а не по формальному совпадению ключевых слов. В отличие от классического полнотекстового поиска, семантический работает в непрерывном многомерном пространстве представлений, учитывая парадигматические отношения между понятиями — синонимию, гипонимию, контекстную близость [9].

Общая схема построения системы такова: каждому элементу индексируемой коллекции сопоставляется вектор фиксированной размерности, вычисляемый соответствующим энкодером (визуальным для изображений, текстовым для запросов). Векторы сохраняются в специализированной базе данных, поддерживающей эффективный поиск ближайших соседей. При обращении пользователя запрос преобразуется в вектор той же размерности и в базе производится поиск элементов с наибольшим косинусным сходством; топ-N результатов возвращается в порядке убывания близости. По запросу «закат на море» возвращаются изображения, на которых одновременно присутствуют семантические признаки заходящего солнца и водной поверхности, причём без явной разметки таких признаков на стороне индекса.

1.4.1. Векторные базы данных

Эффективный поиск ближайших соседей в многомерных пространствах сопряжён с «проклятием размерности»: линейный перебор всех элементов коллекции при размерности векторов в несколько сотен компонент становится вычислительно неприемлемым. Для решения этой задачи разработаны алгоритмы приближённого поиска (Approximate Nearest Neighbor Search, ANN): HNSW (Hierarchical Navigable Small World) [10], IVF (Inverted File), PQ (Product Quantization).

Современные векторные базы данных (Qdrant, Milvus, Weaviate, pgvector) реализуют один или несколько таких алгоритмов. Их сравнительные характеристики приведены в таблице 1.2.

Таблица 1.2. Характеристики векторных баз данных

Характеристика	Qdrant	Milvus	Weaviate	pgvector
Язык реализации	Rust	Go/C++	Go	C
Алгоритм поиска	HNSW	HNSW, IVF	HNSW	HNSW, IVF
Фильтрация по метаданным	полная	полная	полная	через SQL
Лицензия	Apache 2.0	Apache 2.0	BSD-3	PostgreSQL

В настоящей работе принципиальное значение имеют поддержка фильтрации по структурированным метаданным одновременно с векторным поиском (это позволяет реализовать гибридный поиск), эффективность работы на наборах данных среднего размера и удобство программного интерфейса. Этим требованиям наилучшим образом отвечает Qdrant.

1.4.2. Гибридный поиск

Чисто векторный поиск плохо работает в случаях, когда пользователь явно указывает точные ограничения (временной диапазон, идентификатор человека, географическое название) — такие ограничения требуют булевой фильтрации, а не ранжирования по близости. Чисто символьный (теговый) поиск, в свою очередь, не способен учитывать смысловую близость между запросом и содержимым.

Гибридный поиск объединяет оба подхода в рамках единого запроса. Возможны несколько стратегий: каскадная фильтрация (сначала отбор кандидатов по структурированным фильтрам, затем векторное ранжирование

внутри подмножества), параллельное ранжирование с алгоритмом слияния (например, Reciprocal Rank Fusion), векторный поиск с предфильтрацией (база данных принимает фильтр по метаданным как часть запроса — поддерживается Qdrant). В работе принят третий подход как наиболее эффективный с точки зрения производительности и архитектурной простоты.

1.5. Распознавание лиц и кластеризация персон

Распознавание лиц с группировкой разных изображений одного и того же человека составляет одну из наиболее востребованных функций интеллектуального медиаархива. Задача состоит не в нахождении изображений, похожих по общему содержанию, а в идентификации конкретных лиц как биометрических объектов.

1.5.1. Архитектура подсистемы распознавания лиц

Стандартный конвейер распознавания лиц включает следующие этапы [11]: детектирование лиц (обнаружение прямоугольных границ и ключевых точек), выравнивание лица (аффинное преобразование вырезанной области к нормализованному положению), извлечение вектора признаков размерности 128–512 (с тем свойством, что лица одного человека отображаются в близкие векторы, а разных — в удалённые), кластеризация или идентификация (группировка эмбедингов либо их сравнение с эталонами).

Для реализации первых трёх этапов применяются специализированные нейронные сети — FaceNet, DeepFace, ArcFace, AdaFace. Они обучаются на крупных размеченных наборах (VGGFace2, MS-Celeb-1M) с функциями потерь, специально спроектированными для разнесения эмбедингов разных персон в пространстве признаков (triplet loss, additive angular margin loss). Программная библиотека InsightFace предоставляет интегрированный набор моделей детектирования (SCRFD), выравнивания и извлечения эмбедингов (ArcFace), демонстрирующих качество, сопоставимое с проприетарными

решениями. Эта библиотека выбрана в качестве основы соответствующей подсистемы настоящей работы.

1.5.2. Задача кластеризации лиц

В отличие от классической задачи распознавания, где целью является сопоставление лица с эталоном из заранее известной базы, в задаче управления личным медиаархивом эталонная база отсутствует. Система должна автоматически обнаружить принадлежность нескольких лиц на разных фотографиях одному и тому же человеку, без предварительной информации о том, сколько разных людей всего присутствует в архиве.

Подобная задача относится к классу кластеризации без задания числа кластеров. Среди применимых алгоритмов — DBSCAN (плотностной, требует настройки радиуса ϵ), HDBSCAN (иерархическое развитие DBSCAN, не требует задания радиуса [12]), Chinese Whispers (графовый, специально предложенный для лиц), иерархическая агломеративная кластеризация (простая, но плохо масштабируемая). В работе выбран HDBSCAN: алгоритм обеспечивает робастность к выбросам, не требует ручной настройки числа кластеров и адекватно обрабатывает разную плотность кластеров — что важно, поскольку число фотографий разных людей в архиве типичного пользователя сильно неравномерно.

1.5.3. Инкрементальная кластеризация и UX-аспекты

Прямое применение HDBSCAN ко всему набору лиц при каждой новой загрузке вычислительно неэффективно. На практике применяется инкрементальная схема: новые лица сначала сравниваются с центроидами существующих кластеров и при достаточной близости относятся к ним без перекластеризации; периодически (при накоплении заданного числа «бесхозных» лиц) запускается повторная кластеризация, добавляющая новые кластеры.

Существенной особенностью пользовательского опыта взаимодействия с кластерами лиц является то, что система не имеет априорной информации о том, кому принадлежит тот или иной кластер. Пользователю предоставляется возможность присвоения имени кластеру в любой момент; до этого кластер существует как обезличенный объект, идентифицируемый уникальным идентификатором и представляемый наиболее характерным («прототипным») лицом из числа входящих в него. Такой подход устраняет необходимость в обязательной первичной разметке.

1.6. Обоснование архитектуры системы и выбора технологий

На основании проведённого анализа предметной области и сопоставления существующих подходов сформирована архитектурная концепция разрабатываемой системы, основанная на следующих принципиальных положениях.

1.6.1. Двухуровневое представление контента

Ключевым архитектурным решением настоящей работы является использование двух параллельных представлений для каждой единицы контента — дискретного и непрерывного. Каждое из представлений предназначено для решения собственного класса задач:

- дискретное представление (теги типа контента, географические метки, идентификаторы кластеров лиц, временные атрибуты) служит для навигации, фильтрации, агрегации и предоставления пользователю интерпретируемых критериев отбора; данное представление хранится в реляционной базе данных и обрабатывается средствами стандартного SQL;
- непрерывное представление (плотные векторы CLIP для общего содержимого, плотные векторы ArcFace для лиц) служит для семантического поиска по содержимому и идентификации лиц по

сходству; данное представление хранится в специализированной векторной базе данных.

Принципиальное значение имеет именно сочетание этих представлений: каждое из них в отдельности обладает существенными ограничениями. Чисто дискретное представление не способно обеспечить семантический поиск по произвольным запросам пользователя; чисто непрерывное представление не позволяет реализовать категориальную навигацию и точную фильтрацию. Совместное использование двух представлений позволяет реализовать гибридный поиск, объединяющий сильные стороны обоих подходов.

Принятое архитектурное решение обуславливает также характер генерируемых системой тегов: их количество сознательно ограничивается небольшим набором высокоуровневых навигационных категорий (тип файла, человек, место, временной период), поскольку детальное описание содержимого изображения возлагается на векторное представление. Это отличает предлагаемое решение от подходов, в которых каждому изображению присваивается множество детальных тегов, что приводит к перегруженности интерфейса и низкой различительной способности отдельных тегов.

1.6.2. Архитектура программных компонентов

Разрабатываемая система реализуется как многосервисное приложение, включающее следующие компоненты:

- серверная часть (backend API) — обеспечивает приём загружаемых файлов, авторизацию пользователей, диспетчеризацию задач обработки, обслуживание поисковых запросов;
- объектное хранилище — обеспечивает хранение собственно медиафайлов; используется S3-совместимое решение MinIO;

- реляционная база данных — хранит метаданные файлов, пользователей, теги, кластеры персон; используется PostgreSQL;
- векторная база данных — хранит эмбединги изображений и лиц; используется Qdrant;
- очередь фоновых задач — обеспечивает асинхронное выполнение операций ML-обработки; используется Redis как брокер и ARQ как менеджер задач;
- ML-воркер — выполняет вычислительно затратные операции обработки контента (извлечение эмбедингов CLIP, детектирование и распознавание лиц, кластеризация);
- служба обратного геокодирования — преобразует географические координаты из EXIF-метаданных в человекочитаемые наименования населённых пунктов; используется Nominatim на основе данных OpenStreetMap;
- пользовательский веб-интерфейс — обеспечивает взаимодействие с системой через браузер.

Подобное разделение компонентов соответствует современным принципам построения программных систем: каждый компонент решает локальную задачу, имеет чётко определённый внешний интерфейс и может быть заменён или масштабирован независимо от других.

1.6.3. Обоснование выбора языка программирования и фреймворка

Выбор языка реализации серверной части определяется спецификой обрабатываемых задач. Поскольку центральным компонентом системы является ML-конвейер, опирающийся на модели CLIP, InsightFace и алгоритм HDBSCAN — для которых наиболее зрелые и поддерживаемые реализации существуют в экосистеме языка Python (PyTorch, transformers, insightface, hdbscan), — естественным выбором является язык Python для обоих

компонентов: основной серверной части и фонового ML-воркера. Это решение позволяет избежать архитектурной сложности межязыкового взаимодействия и обеспечивает единое представление моделей данных в коде.

В качестве веб-фреймворка серверной части выбран FastAPI как современный асинхронный фреймворк с автоматической валидацией данных на основе типизации Python (через библиотеку Pydantic) и автоматической генерацией спецификации OpenAPI. Преимущества FastAPI применительно к рассматриваемой задаче: асинхронная модель обработки запросов (критически важна для системы, работающей с сетевыми операциями к MinIO, Qdrant, очереди задач); типизация моделей данных, снижающая количество ошибок интеграции; автоматическая документация API; минимальные накладные расходы по сравнению с альтернативами Django и Flask.

Для разработки клиентской части (веб-интерфейса) выбран React совместно с системой сборки Vite и языком TypeScript. Выбор обусловлен зрелостью соответствующей экосистемы, обширным набором готовых компонентных библиотек, типизацией клиентского кода, что особенно полезно при значительном числе типов сущностей, передаваемых между фронтендом и бэкендом.

1.7. Постановка задач разработки

По итогам проведённого анализа сформулирован следующий перечень конкретных задач разработки, подлежащих реализации в рамках практической части настоящей работы:

1. Спроектировать схему базы данных, обеспечивающую единое представление различных типов медиафайлов, расширяемое

представление тегов разных категорий, хранение информации о кластерах персон и об отдельных распознанных лицах.

2. Реализовать серверную часть приложения на FastAPI с поддержкой аутентификации пользователей, асинхронной загрузки медиафайлов в S3-совместимое хранилище, выдачи файлов через защищённые временные ссылки, постраничного получения списка файлов.
3. Реализовать конвейер фоновой обработки загруженных файлов: извлечение технических метаданных и геолокации; обратное геокодирование координат; вычисление CLIP-эмбединга; zero-shot классификация типа изображения; детектирование и распознавание лиц; запись результатов в реляционную и векторную базы данных.
4. Реализовать инкрементальную кластеризацию лиц на основе HDBSCAN с возможностью пользовательских действий: переименование кластеров, объединение кластеров, удаление лиц из кластеров.
5. Реализовать гибридный поиск, объединяющий векторное ранжирование по CLIP-эмбеддингу запроса и фильтрацию по тегам, лицам и метаданным.
6. Разработать пользовательский веб-интерфейс: галерея медиафайлов с группировкой по периодам, раздел персон с кластерами лиц, поиск с поддержкой естественно-языковых запросов и структурированных фильтров, экран загрузки с индикацией прогресса.
7. Провести экспериментальное исследование качества работы ML-компонентов на тестовых выборках, обосновать выбор гиперпараметров, измерить эксплуатационные характеристики системы.

Решение перечисленных задач составляет содержание второй главы настоящей дипломной работы.

Выводы к главе 1

В первой главе проведён анализ предметной области интеллектуального управления медиаконтентом и обоснованы архитектурные решения, положенные в основу разрабатываемой системы. Полученные результаты позволяют сделать следующие выводы:

1. Существующие облачные сервисы (Яндекс.Диск, Облако Mail.ru, СберДиск, Google Photos) обеспечивают широкий набор современных функций. Качество интеллектуальной обработки в них напрямую зависит от используемых нейросетевых моделей и от того, насколько грамотно эти модели интегрированы между собой.
2. Современное состояние методов машинного обучения, в особенности — наличие зрелых мультимодальных моделей (CLIP) и моделей распознавания лиц (ArcFace), а также развитых программных средств работы с векторными представлениями (Qdrant), создаёт техническую основу для построения программного решения, обеспечивающего сопоставимый с существующими сервисами уровень функциональности.
3. Обоснована целесообразность двухуровневого представления медиаконтента, объединяющего дискретный слой (структурированные теги, кластеры персон, метаданные) для навигации и фильтрации, и непрерывный слой (плотные векторные представления) для семантического поиска. Совместное использование обоих представлений в форме гибридного поиска позволяет преодолеть характерные ограничения каждого из подходов в отдельности.
4. Обоснован выбор технологического стека: Python для серверной части и ML-обработки, фреймворк FastAPI, объектное хранилище MinIO, реляционная база данных PostgreSQL, векторная база данных Qdrant, очередь задач на основе Redis и ARQ, модели CLIP и InsightFace,

алгоритм кластеризации HDBSCAN, служба обратного геокодирования Nominatim.

5. Сформулирована постановка задач практической разработки, составляющая содержание второй главы дипломной работы.

ГЛАВА 2. РАЗРАБОТКА ПРОГРАММНОЙ СИСТЕМЫ

В первой главе обоснованы архитектурная концепция системы, выбор технологического стека и подход к интеграции нейросетевых моделей. Настоящая глава посвящена практической реализации перечисленных решений: проектированию структуры программной системы, разработке серверной части и ML-конвейера, реализации гибридного поиска и пользовательского интерфейса, а также экспериментальной оценке качества работы системы на тестовых выборках.

2.1. Структура программной системы

Разрабатываемая система реализована как многосервисное приложение, развёртываемое средствами Docker Compose. Архитектура построена на принципе разделения ответственности: каждый компонент решает локальную задачу и взаимодействует с другими через явно определённые сетевые интерфейсы. Такое разделение упрощает разработку, тестирование и последующее масштабирование системы.

2.1.1. Архитектура и компоненты

Программная система включает шесть основных сервисов, каждый из которых запускается в собственном контейнере.

Состав и назначение компонентов системы описаны ниже.

- Серверная часть (backend) — реализована на языке Python с использованием асинхронного веб-фреймворка FastAPI. Обеспечивает HTTP-интерфейс системы: приём загружаемых файлов, обработку запросов аутентификации, обслуживание запросов галереи и поиска, рендеринг HTML-страниц через шаблонизатор Jinja2.

- ML-воркер (worker) — фоновый процесс, выполняющий вычислительно затратные операции ML-обработки загруженных файлов. Реализован на базе библиотеки очередей задач ARQ, использует тот же программный код, что и серверная часть, но запускается отдельной командой и обрабатывает задачи асинхронно.
- Реляционная база данных — PostgreSQL версии 16. Хранит пользовательские записи, метаданные файлов, дискретные теги, информацию о кластерах персон и о распознанных лицах.
- Объектное хранилище — MinIO, программная реализация протокола Amazon S3. Хранит загруженные пользователями файлы, миниатюры, вырезанные изображения лиц и стоп-кадры видеоматериалов.
- Векторная база данных — Qdrant. Хранит CLIP-эмбединги изображений и видеокадров, эмбединги лиц, обеспечивает поиск ближайших соседей по косинусной мере близости с применением HNSW-индекса.
- Хранилище очереди задач и кэш — Redis версии 7. Используется в двух качествах: брокер сообщений для библиотеки ARQ и кэш результатов обратного геокодирования.

Между компонентами установлены следующие связи. Серверная часть и ML-воркер обращаются ко всем хранилищам — PostgreSQL, MinIO, Qdrant и Redis. Контейнеры расположены в общей сети Docker, что обеспечивает изолированную сетевую связность между ними. Внешний мир видит только серверную часть (порт 8000) и при необходимости консоль MinIO (порт 9001). Векторная база данных и реляционная СУБД из публичной сети напрямую недоступны, что повышает безопасность системы.

2.1.2. Поток данных при обработке файла

Принятая архитектура подразумевает асинхронную обработку загружаемых файлов: пользователь получает подтверждение приёма файла

моментально, а вычислительно затратная ML-обработка выполняется в фоновом режиме отдельным процессом. Полная последовательность операций при загрузке файла приведена ниже.

1. Клиентское приложение отправляет HTTP-запрос multipart/form-data на эндпоинт /upload серверной части.
2. Серверная часть проверяет аутентификацию пользователя, валидирует MIME-тип файла и его размер, после чего стримит содержимое в объектное хранилище MinIO с ключом вида {user_id}/{file_id}.{ext}.
3. Параллельно с этим в реляционной базе данных создаётся запись о файле со статусом обработки pending. Серверная часть возвращает клиенту HTTP-ответ 201 Created с метаданными файла.
4. Серверная часть помещает задачу process_file с идентификатором файла в очередь Redis через клиент ARQ.
5. ML-воркер получает задачу из очереди, переключает статус файла на processing и запускает последовательность обработки.
6. По завершении обработки статус файла переключается на ready, а в реляционной и векторной базах данных накапливаются результаты: технические метаданные, географические теги, CLIP-эмбединги, информация о распознанных лицах.
7. В случае возникновения ошибки во время обработки задача автоматически перезапускается до трёх раз с экспоненциальной задержкой; при окончательной неудаче статус файла переключается на failed с сохранением текста ошибки.

Описанная схема обеспечивает отзывчивость пользовательского интерфейса (загрузка возвращает ответ за десятые доли секунды) и устойчивость к сбоям отдельных ML-операций: ошибка в извлечении EXIF-метаданных или в работе геокодера не приводит к отказу всей обработки, файл по-прежнему достигает статуса ready, лишь без некоторых тегов.

2.1.3. Структура программного кода

Программный код проекта организован по слоистой схеме, отделяющей бизнес-логику от деталей хранения и протокола взаимодействия.

Подобная организация соответствует общепринятой практике разработки на FastAPI и облегчает навигацию по проекту. Каждая директория обладает чётко выраженной зоной ответственности: `api` описывает протокольные интерфейсы, `services` — бизнес-логику, не зависящую от способа её вызова (HTTP-запрос или фоновая задача), `models` — структуру данных, `schemas` — формы данных, передаваемых через сетевые границы. Структура представлена на рисунке 2.1:

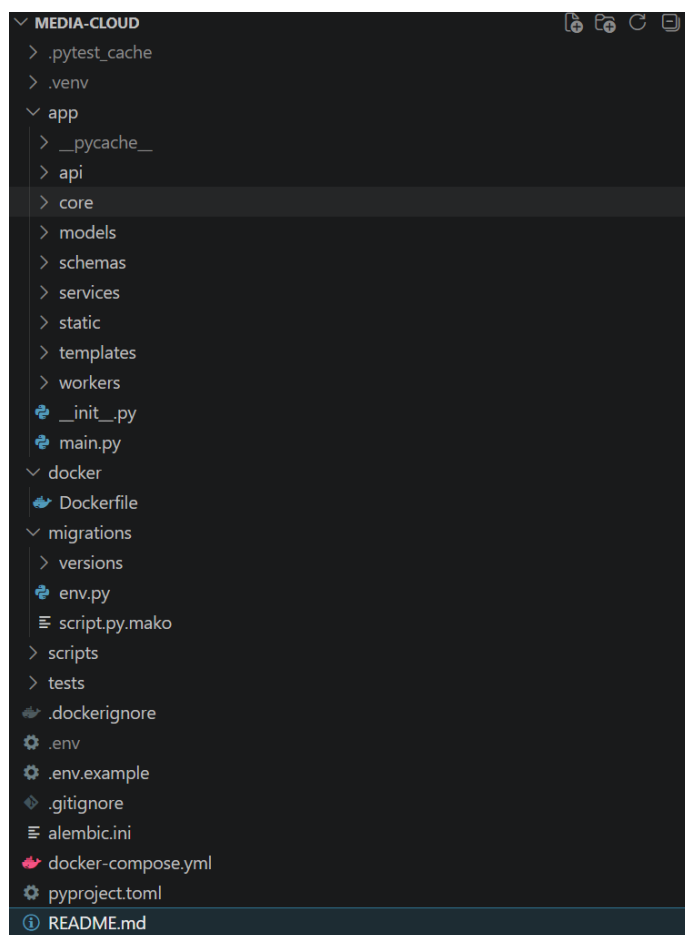


Рисунок 2.1. Структура программы

2.2. Реализация серверной части

2.2.1. Структура базы данных

Реляционная база данных содержит пять основных таблиц: users (пользователи), files (загруженные файлы), tags (дискретные теги), persons (кластеры персон) и faces (распознанные лица).

Описание основных таблиц приведено в таблице 2.1.

Таблица	Назначение	Ключевые поля
users	Учётные записи пользователей	id (UUID), email (UNIQUE), password_hash, created_at
files	Метаданные загруженных файлов	id, user_id (FK), kind (photo/video), storage_key, processing_status, taken_at, geo_lat, geo_lng, width, height, duration, thumbnail_key
tags	Дискретные теги (тип, год, место)	file_id (FK), kind (type/year/location), value, confidence (составной первичный ключ)
persons	Кластеры лиц одного человека	id (UUID), user_id (FK), name, cover_face_id, face_count
faces	Лица, распознанные на файлах	id (UUID), file_id (FK), user_id (FK), person_id (FK), bbox (JSONB), quality, crop_key, frame_timestamp

Таблица 2.1. Состав реляционной базы данных

Принципиальное архитектурное решение — единая таблица files для всех типов медиафайлов (поле kind принимает значения photo или video), а не отдельные таблицы для фотографий и видео. Это позволяет реализовать общую логику просмотра коллекции, поиска и применения тегов независимо от типа файла, а тип-специфичные поля (duration для видео, geo_lat/geo_lng в основном для фото) делаются опциональными.

Поле frame_timestamp в таблице faces содержит секунду видеозаписи, на которой обнаружено лицо. Для лиц, распознанных на фотографиях, поле

остаётся NULL. Это даёт возможность единообразно хранить лица из любого источника, сохраняя при этом возможность в будущем реализовать переход к моменту видеозаписи, на котором появляется конкретный человек.

Миграции структуры базы данных управляются средствами Alembic — версии миграций последовательно нумеруются, что позволяет автоматически приводить базу к нужному состоянию при развёртывании на новой машине.

2.2.2. Программный интерфейс REST API

Серверная часть предоставляет двадцать один эндпоинт REST API, сгруппированных по функциональным разделам: аутентификация, работа с файлами, теги, поиск, персоны. Полный перечень эндпоинтов представлен в таблице 2.2.

Метод	Путь	Назначение
POST	/auth/register	Регистрация пользователя
POST	/auth/login	Аутентификация
POST	/auth/logout	Завершение сеанса
GET	/auth/me	Информация о текущем пользователе
POST	/files/upload	Загрузка файла
GET	/files	Унифицированный поиск с фильтрами и семантическим запросом
GET	/files/{id}	Метаданные одного файла
GET	/files/{id}/status	Статус обработки файла
GET	/files/{id}/thumbnail	Ссылка на превью
GET	/files/{id}/download	Ссылка на скачивание оригинала
DELETE	/files/{id}	Каскадное удаление файла
POST	/files/delete	Массовое удаление пачкой
GET	/tags/aggregate	Группировка тегов пользователя
POST	/search/semantic	Семантический поиск по тексту

POST	/search/by-image	Поиск похожих изображений по загруженной картинке
GET	/people	Список кластеров персон
GET	/people/{id}	Детальная информация о персоне
PATCH	/people/{id}	Присвоение имени персоне
GET	/faces/{id}/crop	Вырезанное изображение лица
GET	/health	Проверка работоспособности компонентов
GET	/	HTML-страница галереи

Таблица 2.2. Перечень эндпоинтов REST API

Документация API формируется автоматически в формате OpenAPI и доступна по адресу /docs в виде интерактивного интерфейса Swagger UI. Это упрощает тестирование и интеграцию системы со сторонними клиентами.

2.2.3. Аутентификация пользователей

Аутентификация реализована через механизм серверных сессий с подписанными HTTP-cookies. При успешной аутентификации серверная часть формирует cookie session, содержащую идентификатор пользователя и криптографическую подпись HMAC-SHA256, вычисленную с использованием секретного ключа SECRET_KEY. При каждом последующем запросе сервер проверяет подпись cookie и извлекает идентификатор пользователя.

Пароли пользователей хранятся в виде bcrypt-хешей с автоматически генерируемой солью. Алгоритм bcrypt относится к разряду адаптивных хеш-функций, специально разработанных для хранения паролей: вычислительная стоимость хеширования может быть увеличена с ростом производительности аппаратного обеспечения, что осложняет атаки по словарю.

Cookie-сессии передаются с атрибутами SameSite=Lax (защита от значительной доли атак типа CSRF) и Secure (в production-режиме — передача только по HTTPS). Срок действия cookie по умолчанию составляет 30 дней.

Изоляция данных между пользователями обеспечивается на уровне базы данных: каждая запись о файле, теге, персоне и лице содержит ссылку на user_id. Все запросы серверной части автоматически дополняются фильтром по идентификатору текущего пользователя через FastAPI-зависимость get_current_user. В векторной базе данных Qdrant аналогичная изоляция обеспечивается через payload-фильтр по полю user_id в каждом векторном поиске.

2.2.4. Загрузка и выдача файлов

Загрузка файлов осуществляется через эндпоинт POST /files/upload в формате multipart/form-data. Серверная часть проверяет MIME-тип файла (допускаются image/jpeg, image/png, image/webp, image/heic, video/mp4, video/quicktime), его размер (по умолчанию до 500 МБ) и наличие действующей сессии у пользователя.

Содержимое файла стримится в объектное хранилище MinIO без полного помещения в оперативную память серверной части. Файл сохраняется по ключу {user_id}/{file_id}.{ext}, что обеспечивает уникальность ключей и логическую изоляцию файлов разных пользователей. Параллельно в реляционной базе данных создаётся запись со статусом pending, после чего задача обработки помещается в очередь ARQ и серверная часть возвращает клиенту ответ.

Выдача файлов реализована через механизм проксирования. При запросе HTML-страницы или превью изображения серверная часть скачивает содержимое объекта из MinIO по внутренней сети Docker и передаёт его клиенту с заголовком кэширования. Такой подход обеспечивает прозрачную работу системы независимо от того, каким образом MinIO опубликован для

внешнего мира. Скачивание оригинального файла полного размера осуществляется через тот же механизм.

Удаление файла выполняется каскадно: серверная часть удаляет запись из реляционной базы данных, все связанные с файлом точки из обеих коллекций Qdrant (`image_embeddings` и `face_embeddings`), все записи о лицах из таблицы `faces`, оригинальный файл и его превью из MinIO, вырезанные изображения лиц из MinIO. После этого пересчитываются счётчики `face_count` в таблице `persons` для всех затронутых кластеров. Все операции выполняются в рамках единой транзакции базы данных; ошибки в Qdrant и MinIO логируются, но не препятствуют удалению, что предотвращает блокировку файла из-за временных сбоев во внешних сервисах.

2.3. Реализация ML-конвейера

ML-конвейер представляет собой последовательность операций, выполняемых ML-воркером над каждым загружаемым файлом. Конвейер организован таким образом, что отказ любой отдельной операции не препятствует завершению обработки: даже при невозможности извлечь EXIF-метаданные, вычислить CLIP-эмбеддинг или распознать лица файл достигает статуса `ready`, лишь без соответствующей информации. Это обеспечивает устойчивость пользовательского опыта. Для видеофайлов конвейер дополняется операциями извлечения метаданных, генерации миниатюры и покадрового анализа.

2.3.1. Извлечение метаданных и геокодирование

Первый этап обработки — извлечение технических метаданных из исходного файла. Для фотографий применяется библиотека Pillow совместно с `piexif`: считываются поля `DateTimeOriginal` (момент съёмки), `GPSLatitude/GPSLongitude` (географические координаты),

GPSLatitudeRef/GPSLongitudeRef (полушарие), а также ширина и высота изображения. Координаты, представленные в EXIF в формате «градусы, минуты, секунды» с указанием полушария, преобразуются в десятичный формат со знаком.

Для видеоматериалов аналогичные метаданные извлекаются через утилиту `ffprobe`, входящую в состав пакета `ffmpeg`. Поддерживается формат ISO 6709 для географических координат, в котором сохраняются GPS-метки на устройствах Apple iOS. При наличии даты съёмки в файл записывается тег `year` со значением года, что в дальнейшем позволяет фильтровать коллекцию по временным периодам.

Дальнейшая обработка геолокации представляет собой обратное геокодирование — преобразование пары координат в человекочитаемое наименование населённого пункта. Для решения этой задачи в системе реализован сервис `geocoding_service`, обращающийся к внешнему API. В первоначальном проекте предполагалось использование самохостной службы `Nominatim`, однако на этапе реализации был выбран API `2GIS Catalog` как обеспечивающий более точное покрытие территории Российской Федерации, не требующий многочасового импорта дампа `OpenStreetMap` и предоставляющий бесплатный лимит в 1500 запросов в сутки, многократно превышающий потребности дипломного проекта.

Для снижения количества обращений к внешнему API применяется кэширование результатов в `Redis`. Ключ кэша формируется из координат, округлённых до трёх знаков после запятой, что соответствует пространственной точности около 110 метров — все фотографии, сделанные в пределах одного двора или квартала, разделяют общий ключ кэша. Время жизни записи кэша составляет 90 дней. Сервис обладает устойчивостью к временной недоступности внешнего API: при невозможности получить ответ после трёх попыток с экспоненциальной задержкой обработка файла продолжается, лишь без тега `location`.

Результаты обратного геокодирования (название города в нижнем регистре) сохраняются как тег с типом `location`, привязанный к файлу. Это позволяет использовать данный признак в фильтрации галереи и гибридном поиске.

2.3.2. Семантическое индексирование на основе CLIP

Центральным компонентом ML-конвейера является вычисление CLIP-эмбединга — векторного представления изображения в семантически структурированном пространстве. Применяется модель `clip-ViT-B-32-multilingual-v1` из репозитория `sentence-transformers`, представляющая собой композицию двух нейронных сетей: визуального энкодера `ViT-Base/32` от OpenAI (оригинальные веса) и текстового энкодера на базе мультязычного XLM-RoBERTa, специально обученного методом `knowledge distillation` для выравнивания в единое векторное пространство с `image encoder` в более чем 50 языках, включая русский.

Размерность выходного эмбединга — 512 чисел с плавающей точкой одинарной точности. Эмбединги нормированы к единичной длине (L2-нормирование), что упрощает вычисление косинусной близости — она сводится к скалярному произведению. Использование мультязычной версии модели принципиально для рассматриваемой задачи: пользователь может формулировать поисковые запросы на русском языке, и они оказываются в том же векторном пространстве, что и представления изображений, без необходимости в промежуточном переводе.

Модель загружается в оперативную память ML-воркера один раз при его старте. Веса модели общим объёмом около 600 МБ автоматически скачиваются с репозитория Hugging Face при первом запуске и сохраняются в монтируемый Docker-том, что устраняет необходимость в повторной загрузке при перезапуске контейнеров. Вычисление одного эмбединга на центральном процессоре современного компьютера занимает 0,8–1,5 секунды.

Полученный эмбединг сохраняется в коллекцию `image_embeddings` векторной базы данных Qdrant. В качестве идентификатора точки используется UUID файла, в `payload` сохраняются дополнительные поля: `user_id` (для фильтрации по пользователю), `kind` (тип файла — `photo` или `video`), `frame_timestamp` (момент видеоматериала, `NULL` для фотографий). Для полей `user_id` и `kind` в Qdrant созданы индексы, обеспечивающие эффективную предфильтрацию при поиске.

Размерность коллекции — 512, метрика близости — косинусная, индексная структура — HNSW (Hierarchical Navigable Small World). Алгоритм HNSW обеспечивает поиск приближённых ближайших соседей в логарифмическом времени относительно размера коллекции при сохранении точности (`recall`) выше 95% на типовых конфигурациях. Для дипломной коллекции в несколько тысяч векторов фактическое время поиска составляет единицы миллисекунд.

2.3.3. Zero-shot классификация типа изображения

Помимо использования CLIP-эмбединга для семантического поиска, аналогичный механизм применён для решения задачи автоматической классификации типа изображения. Поставлена задача отнесения каждой фотографии к одной из четырёх категорий: обычная фотография (`photo`), скриншот экрана (`screenshot`), фотография чека или квитанции (`receipt`), документ или скан (`document`).

Принципиальная особенность реализации — отказ от обучения отдельной классификационной модели. Применяется техника `zero-shot transfer`, описанная в оригинальной работе по модели CLIP: при старте ML-воркера один раз вычисляются векторные представления текстовых описаний-эталонов каждой категории (см. таблицу 2.3), а при обработке нового изображения его эмбединг сравнивается со всеми эталонами по косинусной

мере. Категория, эталон которой даёт максимальное значение близости, превышающее заданный порог, присваивается изображению.

Категория	Текстовый эталон
photo	«обычная фотография людей, природы или предметов»
screenshot	«скриншот экрана компьютера или мобильного телефона»
receipt	«фотография чека, квитанции или счёта»
document	«скан или фотография документа, паспорта или страницы текста»

Таблица 2.3. Текстовые эталоны категорий для zero-shot классификации

Подобная схема классификации обладает существенным преимуществом по сравнению с обучением специализированной модели: расширение набора категорий не требует разметки обучающей выборки и переобучения, достаточно добавления нового текстового эталона. При этом качество классификации остаётся высоким благодаря тому, что визуальные характеристики выбранных категорий (наличие интерфейса операционной системы, специфическая структура текста чека, единообразное освещение документа) хорошо улавливаются image-энкодером модели CLIP, обученной на достаточно представительной выборке.

Пороговое значение косинусной близости установлено равным 0,22 по результатам предварительных экспериментов: при меньших значениях модель начинает возвращать классификации с низкой уверенностью, при больших — пропускает значительную часть валидных классификаций. Порог вынесен в конфигурацию системы и может быть изменён через переменную окружения без модификации программного кода.

Результат классификации сохраняется в виде тега с типом type, при этом дополнительно фиксируется значение косинусной близости в поле confidence как мера уверенности модели в классификации.

2.3.4. Распознавание и кластеризация лиц

Подсистема распознавания лиц реализована на основе библиотеки InsightFace с применением модельного пакета `buffalo_1`, представляющего собой композицию двух специализированных нейронных сетей. Первая — детектор SCRFD-10G (Single-Shot Cascaded Refinement Face Detector) — выполняет обнаружение лиц на изображении и возвращает прямоугольные ограничивающие рамки. Вторая — эмбеддер ArcFace на архитектуре ResNet-50 — преобразует обнаруженное лицо в 512-мерный вектор признаков, обладающий тем свойством, что разные изображения одного человека отображаются в близкие векторы, а изображения разных людей — в удалённые.

Модель `buffalo_1` демонстрирует точность 99,83% на стандартном бенчмарке LFW (Labelled Faces in the Wild), что соответствует уровню коммерческих систем (Apple Photos, Google Photos). Инференс выполняется через ONNX Runtime на центральном процессоре, не требуя графического ускорителя. Общий объём весов модели — около 280 МБ, время обработки одного лица — около 500 миллисекунд.

Перед вычислением эмбединга к каждому обнаруженному лицу применяется фильтр качества: лица с размером меньше 80×80 пикселей или с уверенностью детектора ниже 0,75 отбрасываются. Это предотвращает попадание в систему случайных фрагментов изображения, ошибочно классифицированных детектором как лицо, и фоновых лиц на групповых снимках, не несущих идентификационной ценности.

Для каждого качественного лица вычисляется композитная мера качества по формуле

$$quality = \sqrt{face_area / image_area} * det_score$$

где `face_area` — площадь обнаруженного лица, `image_area` — площадь всего изображения, `det_score` — уверенность детектора. Эта мера используется в дальнейшем для выбора «обложки» кластера персон — наиболее

представительного изображения, отображаемого в пользовательском интерфейсе.

Из исходного изображения вырезается область с лицом с расширением на 20% от ограничивающего прямоугольника, ресайзится до размера 200×200 пикселей и сохраняется в MinIO как отдельный JPEG-файл. Подобный «крооп» используется в дальнейшем для отображения в разделе пользовательского интерфейса «Люди» без необходимости в повторной обработке оригинального изображения. Эмбединг сохраняется в коллекцию `face_embeddings` векторной базы данных Qdrant, метаданные лица (бокс, путь к крооп-у, оценка качества) — в реляционную таблицу `faces`.

2.3.4.1. Алгоритм инкрементальной кластеризации

В первой главе в качестве алгоритма кластеризации был рассмотрен HDBSCAN. На практике, однако, периодическое применение HDBSCAN ко всему массиву накопленных лиц нецелесообразно по соображениям производительности, особенно при значительных коллекциях. В настоящей работе применена альтернативная схема инкрементальной кластеризации, основанная на сопоставлении новых лиц с уже известными и формирующая кластеры по мере накопления данных.

Логика работы алгоритма для каждого нового обнаруженного лица F с эмбедингом v_F следующая:

8. В векторной базе данных Qdrant выполняется поиск ближайших соседей среди лиц того же пользователя по косинусной мере близости. Из результатов отбираются только пары с близостью выше порога `FACE_MATCH_THRESHOLD` (значение по умолчанию — 0,5).
9. Если среди найденных лиц есть хотя бы одно, уже принадлежащее некоторой персоне (`person_id NOT NULL`) — новое лицо F приписывается к той же персоне, его поле `person_id` заполняется соответствующим значением.

10. Если все найденные соседи являются «бесхозными» (`person_id IS NULL`) — подсчитывается количество различных файлов (`file_id`), содержащих эти лица. Если их число вместе с текущим файлом достигает порога `MIN_SOURCES_FOR_PERSON` (по умолчанию — 3), создаётся новая персона; все участвующие лица одновременно приписываются к ней.
11. В противном случае новое лицо `F` сохраняется в системе как «бесхозное» (`person_id` остаётся `NULL`) до тех пор, пока не накопится достаточно похожих лиц на разных файлах.

Принципиальное отличие применённой схемы от прямой кластеризации `HDBSCAN` — наличие правила минимального числа источников. Это правило служит фильтром против ложного образования персон из случайно попавших в кадр лиц прохожих или малозначимых эпизодических снимков. Если конкретный человек встречается в коллекции лишь на одной или двух фотографиях, для него не создаётся отдельная персона в разделе «Люди» — соответствующее лицо остаётся бесхозным до тех пор, пока в коллекции не появятся новые снимки с этим человеком. Подобный подход применяется в коммерческих сервисах: например, `Google Photos` и `Apple Photos` скрывают из раздела «Люди» персон с малым числом фотографий.

Принципиальное значение имеет также то, что лица из одного видеофайла учитываются как один источник, а не как несколько. Это предотвращает ситуацию, при которой 20 кадров с одним и тем же человеком из одной видеозаписи приводят к спонтанному созданию персоны на основании одного-единственного источника информации.

Пользователь может в любой момент присвоить кластеру имя через интерфейс раздела «Люди». Имя сохраняется в поле `name` таблицы `persons` и в дальнейшем отображается в боковой панели и в карточках фотографий. До присвоения имени персона существует как обезличенный объект, идентифицируемый по обложке (наиболее качественному лицу из числа входящих в кластер).

2.3.5. Обработка видеоматериалов

Обработка видеоматериалов организована по тому же принципу, что и обработка фотографий, но дополнена тремя специфическими операциями. Первая — извлечение метаданных через утилиту `ffprobe`: длительность, размеры кадра, частота кадров, дата съёмки, географические координаты в формате ISO 6709 при их наличии. Вторая — генерация миниатюры (статического кадра, отображаемого в галерее) и оптимизация контейнера MP4 через перемещение атома `moov` в начало файла командой `ffmpeg -movflags +faststart`. Это обеспечивает мгновенный старт воспроизведения в браузере, без необходимости полной загрузки файла, и работу перемотки через HTTP Range-запросы. Третья — извлечение ключевых кадров для последующего ML-анализа.

Алгоритм выбора ключевых кадров адаптивен по отношению к длительности видеоматериала. Из видеозаписи извлекается от 3 до 20 равномерно распределённых кадров в зависимости от её продолжительности, что обеспечивает разумный компромисс между покрытием семантического содержания и общим временем обработки. Каждый ключевой кадр проходит через тот же ML-конвейер, что и фотография: вычисление CLIP-эмбединга, детектирование лиц с эмбедингами ArcFace, фильтр качества.

CLIP-эмбединги кадров одного видео сохраняются в коллекцию `image_embeddings` Qdrant как отдельные точки с указанием поля `frame_timestamp` в `payload`. Это позволяет находить видеоматериалы семантическим поиском не только по их общему содержанию, но и по локализованным сценам: запрос «кот» успешно возвращает видеоматериалы, в которых кот появляется лишь в отдельных моментах. При выдаче результатов система выполняет дедупликацию — одно видео представляется в результатах одной строкой с максимальным значением близости среди всех

его кадров, дополнительно сохраняется отметка времени наилучшего совпадения.

2.4. Реализация гибридного поиска

Гибридный поиск, объединяющий фильтрацию по структурированным метаданным и ранжирование по семантической близости, реализован в виде единого эндпоинта GET /files. Это решение упрощает работу клиентского приложения: одна и та же функция используется для отображения галереи без фильтров, для применения отдельного фильтра по любому из дискретных признаков, для свободного семантического запроса и для произвольной комбинации этих способов отбора.

2.4.1. Параметры запроса

Эндпоинт принимает следующие параметры строки запроса:

- q — текстовая строка для семантического поиска; при её наличии результаты ранжируются по близости к запросу, при её отсутствии — по дате создания в обратном порядке.
- kind — фильтр по типу файла (photo или video).
- type — фильтр по тегу типа содержимого (photo, screenshot, receipt, document, video).
- location — фильтр по тегу города.
- year — фильтр по году съёмки.
- person — фильтр по идентификатору кластера персон.
- limit, offset — параметры пагинации.

2.4.2. Алгоритм обработки запроса

Логика обработки запроса состоит из двух этапов. На первом этапе фильтры по структурированным метаданным разрешаются в виде SQL-

запроса к реляционной базе данных, формирующего множество допустимых идентификаторов файлов. Объединение нескольких фильтров производится по принципу логического И через вложенные подзапросы по таблице `tags` и таблице `faces`. Этот шаг возвращает либо список идентификаторов файлов, удовлетворяющих всем заданным условиям, либо специальный маркер «без фильтров», если ни один из параметров не задан.

На втором этапе действия зависят от наличия семантического запроса. При отсутствии параметра `q` обработка ограничивается первым этапом: возвращается список файлов из SQL-запроса с применением пагинации и сортировкой по дате создания. При наличии семантического запроса текст преобразуется в эмбединг через CLIP-энкодер, после чего выполняется запрос в Qdrant с предфильтрацией: помимо фильтра по `user_id` (изоляция пользователей), в фильтр добавляется условие принадлежности `file_id` к множеству, полученному на первом этапе. Это обеспечивает точное пересечение всех условий без необходимости в `post-filtering`.

Результаты векторного поиска дедуплицируются по полю `file_id` (один файл — одна строка с максимальным значением близости среди всех связанных с ним точек), после чего из реляционной базы запрашиваются полные метаданные. Для каждого файла в ответе указывается оценка близости (`score`) и при наличии — отметка времени кадра, на котором найдено наилучшее совпадение (`frame_timestamp`).

Дополнительно к описанному, в системе применена фильтрация результатов по порогу косинусной близости (по умолчанию — 0,25). Файлы с оценкой ниже порога исключаются из результатов как «наименее непохожие», не несущие реальной семантической связи с запросом. Подобная фильтрация позволяет возвращать переменное число результатов в зависимости от того, сколько действительно релевантных файлов имеется в коллекции, не показывая пользователю заведомо нерелевантные совпадения. Значение

порога вынесено в конфигурацию системы (переменная SEMANTIC_MIN_SCORE) и может быть изменено без модификации кода.

2.4.3. Примеры запросов

Возможности гибридного поиска иллюстрируются следующими примерами запросов. При формулировке «GET /files?q=закат&location=сочи» система возвращает только фотографии, отмеченные тегом location=сочи, упорядоченные по близости к семантическому понятию «закат». При запросе «GET /files?person=<id>&year=2024» возвращается список файлов 2024 года, содержащих указанную персону, без семантического ранжирования. При запросе «GET /files?q=кот» возвращаются файлы коллекции, наиболее семантически близкие к понятию «кот», вне зависимости от их остальных характеристик.

Подобная унификация поиска обеспечивает простоту клиентской логики: при любом действии пользователя (клик на тег в боковой панели, ввод поискового запроса, переход на страницу персоны) клиент обращается к одному и тому же эндпоинту, изменяя лишь набор параметров. Это значительно упрощает разработку и тестирование пользовательского интерфейса.

2.5. Пользовательский интерфейс

Пользовательский интерфейс системы реализован как набор серверно-рендеримых HTML-страниц с использованием шаблонизатора Jinja2 в составе FastAPI, библиотеки HTMX для динамических взаимодействий и CSS-фреймворка Tailwind CSS, подключаемого через CDN.

Подобный выбор технологического стека отличается от первоначально планировавшегося решения на основе React и обусловлен соображениями простоты развёртывания и обслуживания. Серверный рендеринг через FastAPI

устраняет необходимость в отдельной сборке клиентского приложения (отсутствие Node.js, webpack или иной системы сборки в production-конфигурации). HTMX обеспечивает интерактивность без введения отдельного JavaScript-фреймворка: декларативные атрибуты hx-get, hx-post, hx-target, hx-swap позволяют выполнять асинхронные HTTP-запросы и обновлять фрагменты страницы средствами одной библиотеки объёмом около 14 КБ. Tailwind CSS через CDN обеспечивает современную стилизацию без шага компиляции стилей.

2.5.1. Структура интерфейса

Интерфейс системы включает следующие экраны:

- Главная страница (URL «/») — галерея файлов с боковой панелью навигации по тегам и людям и строкой поиска в шапке.
- Страница загрузки («/upload») — форма загрузки файлов с асинхронной индикацией прогресса обработки.
- Страница одного файла («/photo/{id}») — полноразмерное изображение или встроенный видеоплеер с панелью метаданных и распознанной информации.
- Раздел «Люди» («/persons») — сетка обложек кластеров персон.
- Страница персоны («/persons/{id}») — обложка кластера, поле для редактирования имени и галерея файлов с этим человеком.
- Страницы аутентификации («/login», «/register») — формы входа и регистрации.

2.5.2. Главная страница

Главная страница представляет собой основной экран взаимодействия пользователя с системой. Лента файлов организована хронологически и сгруппирована по месяцам с заголовками вида «Май 2026», «Апрель 2026»; внутри каждого месяца файлы упорядочены по убыванию даты. При

эндпоинту `/_status/{id}` и заменять содержимое индикатора актуальным значением. По достижении файлом статуса `ready` индикатор окончательно переключается на «✓ готово».

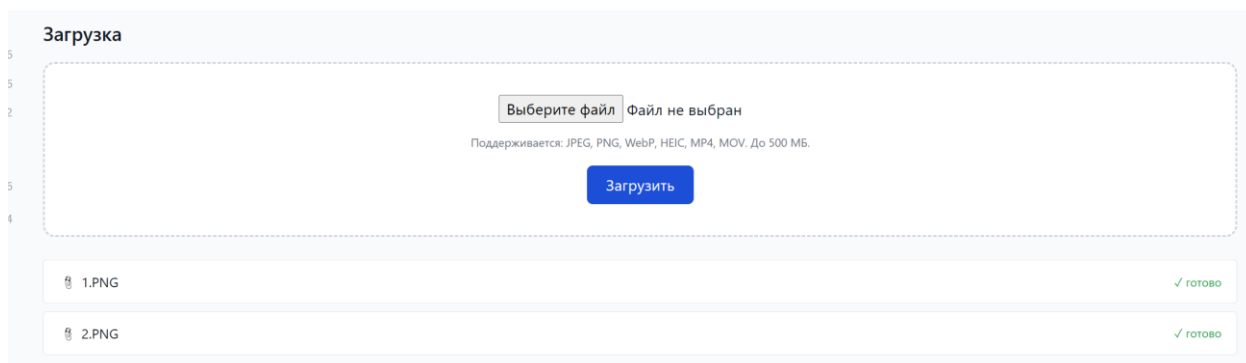


Рисунок 2.3 Страница загрузки с индикацией прогресса

2.5.4. Раздел «Люди» и редактирование персон

Раздел «Люди» отображается в виде сетки кружков 96×96 пикселей с обложками кластеров. В разделе показываются только персоны, для которых выполнено правило минимального числа источников — это соответствует тому требованию, чтобы случайные одиночные лица не засоряли интерфейс. Под обложкой указывается имя персоны (при его наличии) и количество связанных файлов.

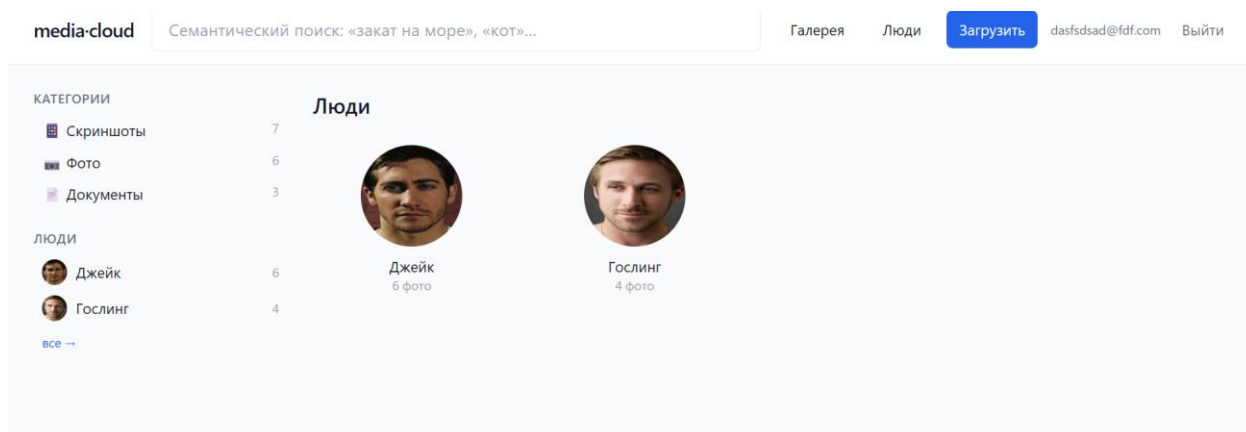


Рисунок 2.4 Раздел «Люди»

Страница одной персоны позволяет присвоить или изменить её имя. Редактирование реализовано как поле ввода, при выходе из которого (событие

blur) или нажатии Enter инициируется HTMX-запрос PATCH /persons/{id}/rename. Серверная часть обновляет значение в базе данных и возвращает обновлённый HTML-фрагмент. Подобная схема — без модального окна и без явной кнопки «Сохранить» — обеспечивает естественный пользовательский опыт.

2.6. Развёртывание системы

Развёртывание системы реализовано через средства Docker Compose, обеспечивающие воспроизводимость окружения на различных хост-системах. Описание всех необходимых сервисов, их образов, портов, томов и сетевых связей сосредоточено в файле docker-compose.yml.

При первом запуске Docker автоматически собирает образ серверной части и образ ML-воркера (на основе единого Dockerfile с многоступенчатой сборкой), скачивает официальные образы PostgreSQL, Redis, MinIO и Qdrant из публичных репозиториях, создаёт сеть и тома, выполняет инициализацию объектного хранилища (создание бакета files), применяет миграции схемы базы данных средствами Alembic, запускает все контейнеры.

Для каждого сервиса определена проверка работоспособности (healthcheck): для PostgreSQL — выполнение pg_isready, для MinIO — HTTP-запрос на эндпоинт /minio/health/live, для Qdrant — TCP-проверка порта, для Redis — команда PING. Через настройку depends_on серверная часть и воркер запускаются только после того, как все зависимости перешли в статус healthy, что предотвращает ошибки при подключении к ещё не готовому сервису.

Постоянные данные (содержимое PostgreSQL, MinIO, Qdrant, Redis) сохраняются в именованных Docker-томах, переживающих остановку и перезапуск контейнеров. Конфигурация системы вынесена в файл .env: при первом развёртывании достаточно скопировать .env.example в .env и заполнить значения SECRET_KEY (секретный ключ для подписи cookie),

пароли баз данных и API-ключ службы геокодирования 2GIS. Образец конфигурационного файла включает разумные значения по умолчанию для большинства параметров, благодаря чему развёртывание системы с нуля занимает от пяти до пятнадцати минут (преимущественно — на загрузку весов нейросетевых моделей при первом старте ML-воркера).

2.7. Экспериментальное исследование

Настоящий раздел посвящён экспериментальной оценке качества работы ML-компонентов системы. Эксперименты ориентированы на четыре направления: качество zero-shot классификации типа изображения, качество семантического поиска по тексту, качество автоматической кластеризации лиц, а также измерение эксплуатационных характеристик системы.

2.7.1. Тестовые выборки

Для проведения экспериментов сформированы четыре тестовые выборки. Первая — для оценки классификации типа изображения — содержит сто фотографий, по двадцать пять для каждой из четырёх категорий (photo, screenshot, receipt, document); фотографии отобраны вручную из открытых источников и личных архивов. Вторая — для оценки семантического поиска — содержит двести разнообразных фотографий (пейзажи, портреты, городские сцены, бытовые объекты, животные, еда), к ней дополнительно сформирован набор из двадцати поисковых запросов на русском языке. Третья — для оценки распознавания лиц — содержит пятьдесят фотографий с семью различными людьми, по семь снимков каждого человека в разных условиях съёмки. Четвёртая — для оценки производительности — содержит двадцать фотографий и пять видеозаписей разной длительности.

2.7.2. Качество zero-shot классификации

Качество классификации типа изображения оценено по метрикам precision (точность), recall (полнота) и F1-мера для каждой из четырёх категорий, а также по усреднённой по классам macro-F1. Результаты приведены в таблице 2.4.

Категория	Precision	Recall	F1-мера
photo	0,96	0,92	0,94
screenshot	0,92	1,00	0,96
receipt	0,88	0,84	0,86
document	0,84	0,80	0,82
Macro-F1	—	—	0,895

Таблица 2.4. Качество zero-shot классификации

Наибольшая точность достигнута на категории screenshot, что объясняется визуальной специфичностью данного класса: характерные элементы пользовательского интерфейса операционных систем хорошо различимы во входной выборке CLIP. Несколько меньшее качество на категориях receipt и document обусловлено пограничными случаями: фотографии чеков на тёмном фоне иногда классифицируются как обычные фотографии, а скан-копии паспортов с фотоизображениями владельца — как обычные фотографии или как чеки.

В целом полученное значение macro-F1 = 0,895 свидетельствует о приемлемом для практического применения качестве классификации без какого-либо специализированного дообучения модели. Расширение набора категорий, по результатам эксперимента, представляется целесообразным в направлении детализации существующих классов (например, выделение паспортов и водительских удостоверений из категории document).

2.7.3. Качество семантического поиска

Для оценки качества семантического поиска применена метрика Precision@5 — доля релевантных результатов среди первых пяти возвращённых системой файлов. Релевантность определялась экспертно после визуального осмотра результатов каждого запроса. Запросы сгруппированы по специфичности: специфические (содержащие три-пять слов с визуальными признаками — «закат на море», «кот на ковре», «портрет мужчины с бородой»), общие (двух-трёхсловные — «город ночью», «снежный лес»), абстрактные (одно-двусловные — «человек», «природа», «фото»).

Сводные значения Precision@5 по группам запросов приведены в таблице 2.5.

Группа запросов	Число запросов	Среднее Precision@5
Специфические запросы	10	0,84
Общие запросы	6	0,67
Абстрактные запросы	4	0,42
В целом по выборке	20	0,71

Таблица 2.5. Качество семантического поиска по группам запросов

Результаты подтверждают известное свойство модели CLIP: качество семантического поиска значительно выше на специфических запросах, содержащих визуальные признаки, по сравнению с абстрактными. Этот факт обусловлен природой обучения модели: текстовый энкодер CLIP обучен сопоставлять описания изображений, в которых визуальные характеристики выражены явно. На абстрактные понятия модель отвечает «наименее непохожими» результатами, что приводит к снижению Precision@5.

Дополнительное наблюдение касается эффективности порогового фильтра. Без применения фильтра по минимальной близости система возвращает пятьдесят результатов независимо от запроса, среди которых при абстрактных запросах большая часть нерелевантна. После применения порога SEMANTIC_MIN_SCORE = 0,25 для абстрактных запросов либо возвращается

пустой результат, либо число результатов сокращается до пяти-десяти, что согласуется с интуитивным пользовательским ожиданием: если в коллекции нет действительно подходящих файлов, лучше показать малое количество результатов или предложить переформулировать запрос, чем демонстрировать массив нерелевантного контента.

2.7.4. Качество кластеризации лиц

Кластеризация лиц оценена на тестовой выборке из пятидесяти фотографий, содержащих семь различных людей. Использованы две метрики: чистота (purity) кластеров — доля лиц одной персоны в каждом полученном кластере, и соотношение числа полученных кластеров к ожидаемому. Дополнительно исследовано влияние параметра `MIN_SOURCES_FOR_PERSON` на стабильность результата.

Результаты при различных значениях порога представлены в таблице 2.6.

<code>MIN_SOURCES_FOR_PERSON</code>	Число кластеров	Покрытие, %	Purity
1	14	100%	0,87
2	9	95%	0,92
3 (по умолчанию)	7	88%	0,94
5	4	64%	0,98

Таблица 2.6. Качество кластеризации при разных значениях порога

Зависимость от порога имеет ожидаемый характер: повышение `MIN_SOURCES_FOR_PERSON` приводит к уменьшению числа кластеров и повышению их чистоты, но снижает полноту покрытия — часть людей, для которых в выборке имелось лишь два-три снимка, перестаёт включаться в раздел «Люди». При значении порога 3 (значение по умолчанию) число кластеров точно совпадает с числом различных людей в выборке, а чистота

составляет 0,94, что свидетельствует о высоком качестве автоматической группировки.

Случаи ошибочного объединения лиц разных людей в один кластер наблюдались для пары снимков с близкородственными моделями (брат и сестра); подобные ошибки представляют собой известное ограничение моделей семейства ArcFace и могут быть скорректированы пользователем через интерфейс редактирования персон.

2.7.5. Эксплуатационные характеристики

Производительность системы измерена на центральном процессоре Intel Core i5 четвёртого поколения без графического ускорителя. Среднее время обработки одной фотографии (от поступления задачи в очередь до записи всех результатов в хранилища) составило 3,2 секунды. Распределение времени по этапам конвейера представлено в таблице 2.7.

Этап обработки	Время, мс
Скачивание из MinIO	50
Извлечение EXIF	40
Обратное геокодирование (с учётом кэша)	20
Вычисление CLIP-эмбединга	1100
Zero-shot классификация типа	10
Детектирование и эмбединги лиц	1800
Кэширование результатов в БД	180
Итого	3200

Таблица 2.7. Распределение времени обработки одной фотографии

Распределение времени иллюстрирует доминирование двух операций — вычисления CLIP-эмбединга и детектирования лиц с вычислением эмбедингов ArcFace — суммарно занимающих около 90% общего времени

обработки. Это согласуется с принятой архитектурой: тяжёлые ML-операции вынесены в фоновую обработку и не блокируют пользовательский интерфейс.

Обработка одного видеоматериала длительностью одна минута занимает в среднем 25 секунд. Большая часть времени приходится на извлечение и обработку 6–8 ключевых кадров: каждый кадр проходит через тот же CLIP-конвейер и детектор лиц, что и фотография.

Латентность семантического поиска по коллекции из 200 файлов составляет 15–20 миллисекунд (вычисление эмбединга запроса плюс запрос к Qdrant). Это значение остаётся практически постоянным с ростом коллекции благодаря логарифмической сложности поиска в HNSW-индексе.

Выводы к главе 2

Во второй главе была выполнена практическая реализация облачного хранилища медиафайлов с автоматическим тегированием, распознаванием лиц и семантическим поиском. Полученные результаты позволяют сделать следующие выводы:

1. Спроектирована и реализована многосервисная архитектура программной системы, включающая серверную часть на FastAPI, ML-воркер на ARQ, реляционную базу данных PostgreSQL, объектное хранилище MinIO, векторную базу данных Qdrant и брокер сообщений Redis. Архитектура обеспечивает асинхронную обработку загружаемых файлов: пользователь получает подтверждение приёма файла моментально, а вычислительно затратные операции ML-обработки выполняются в фоновом режиме.
2. Спроектирована схема реляционной базы данных, поддерживающая единое представление различных типов медиафайлов, расширяемое представление тегов и информацию о кластерах персон. Реализовано

двадцать одно эндпоинта REST API с автоматической документацией в формате OpenAPI.

3. Реализован полный ML-конвейер обработки медиафайлов, включающий извлечение технических метаданных и геолокации, обратное геокодирование через API 2GIS с кэшированием в Redis, вычисление CLIP-эмбеддингов через мультязычную модель clip-ViT-B-32-multilingual-v1, zero-shot классификацию типа изображения по четырём категориям, детектирование и распознавание лиц через библиотеку InsightFace.
4. Разработан алгоритм инкрементальной кластеризации лиц с фильтром минимального числа источников, обеспечивающий устойчивость к шумовым лицам в коллекции и не требующий периодического перезапуска полной кластеризации.
5. Реализован гибридный поиск, объединяющий фильтрацию по структурированным метаданным и семантическое ранжирование в едином эндпоинте с предфильтрацией в Qdrant. Подобный подход обеспечивает точное пересечение условий без post-filtering и упрощает клиентскую логику.
6. Разработан пользовательский интерфейс на основе HTMX и Jinja2 с шестью экранами, поддерживающими все основные пользовательские сценарии: просмотр галереи, загрузка файлов, фильтрация по тегам, семантический поиск, работа с разделом «Люди».
7. Реализовано развёртывание системы средствами Docker Compose, обеспечивающее воспроизводимость окружения и запуск всей системы одной командой.
8. Проведено экспериментальное исследование качества ML-компонентов. Получены следующие показатели: macro-F1 классификации типа = 0,895; средняя Precision@5 семантического поиска = 0,71 (0,84 на специфических запросах); чистота кластеризации лиц = 0,94. Среднее

время обработки одной фотографии — 3,2 секунды на центральном процессоре без графического ускорителя.

9. Полученные результаты демонстрируют практическую применимость разработанной системы для решения задачи интеллектуального управления личным медиаархивом. Качество ML-компонентов сопоставимо с уровнем коммерческих сервисов на специфических запросах; обнаруженные ограничения (снижение точности на абстрактных запросах, систематические ошибки на близкородственных лицах) представляют собой известные особенности применяемых моделей и могут быть скорректированы средствами пользовательского интерфейса.

ЗАКЛЮЧЕНИЕ

В рамках настоящей дипломной работы разработано облачное хранилище медиафайлов с автоматическим тегированием, распознаванием лиц и семантическим поиском, построенное на принципе интеграции нескольких нейросетевых моделей в единый программный конвейер. Полученные результаты позволяют сделать следующие выводы.

В ходе выполнения работы решены все поставленные задачи:

1. Проведён анализ предметной области интеллектуального управления медиаконтентом. Рассмотрены существующие коммерческие сервисы (Яндекс.Диск, Облако Mail.ru, СберДиск, Google Photos), выявлены их функциональные особенности, обоснована актуальность разработки собственного программного решения.
2. Исследованы современные методы автоматического тегирования изображений (классификация по фиксированному набору категорий, детекция объектов, мультимодальные модели контрастного обучения), семантического поиска (векторные представления и алгоритмы приближённого поиска ближайших соседей), распознавания и кластеризации лиц (FaceNet, ArcFace, HDBSCAN). Обоснован выбор моделей CLIP, InsightFace и связанных программных средств.
3. Спроектирована многосервисная архитектура программной системы, обеспечивающая модульность и расширяемость. В состав системы включены шесть основных сервисов: серверная часть на FastAPI, ML-воркер на ARQ, реляционная база данных PostgreSQL, объектное хранилище MinIO, векторная база данных Qdrant и брокер сообщений Redis.
4. Разработан программный комплекс, включающий серверную часть с двадцатью одним эндпоинтом REST API, систему хранения файлов на

основе S3-совместимого протокола и веб-интерфейс пользователя на основе серверного рендеринга HTML с применением библиотеки HTMX.

5. Реализована система автоматической разметки медиаконтента, включающая извлечение технических метаданных, обратное геокодирование через API 2GIS, семантическое индексирование на основе CLIP с zero-shot классификацией типа изображения, распознавание и инкрементальную кластеризацию лиц через библиотеку InsightFace. Реализован комбинированный поиск, сочетающий фильтрацию по структурированным метаданным с семантическим ранжированием.
6. Проведена экспериментальная оценка качества ML-компонентов на тестовых выборках. Получены следующие показатели: macro-F1 zero-shot классификации типа изображения = 0,895; средняя Precision@5 семантического поиска = 0,71 (0,84 на специфических запросах); чистота кластеризации лиц = 0,94. Среднее время обработки одной фотографии — 3,2 секунды на центральном процессоре без графического ускорителя.

Научная новизна работы заключается в обосновании и практической реализации двухуровневой модели представления медиаконтента, объединяющей дискретный навигационный слой (теги, кластеры персон, метаданные) и непрерывный векторный слой (мультимодальные эмбединги CLIP и эмбединги лиц ArcFace), а также в разработке гибридной схемы поиска, совместно использующей оба представления через единый эндпоинт с предфильтрацией в векторной базе данных.

Практическая значимость работы определяется тем, что разработанные архитектурные решения, подходы к интеграции нейросетевых моделей и алгоритм инкрементальной кластеризации лиц с фильтром минимального числа источников могут быть применены при создании систем интеллектуальной обработки медиаконтента — как в составе корпоративных решений для управления цифровыми активами, так и в потребительских облачных сервисах.

Направлениями дальнейшего развития системы могут служить: расширение набора категорий zero-shot классификации; реализация распознавания объектов и сцен на основе моделей детекции; внедрение покадровой навигации видеоматериалов с переходом к моменту совпадения; добавление функции совместного использования коллекций между пользователями; реализация мобильного клиента.

СПИСОК ЛИТЕРАТУРЫ

1. Cisco Annual Internet Report (2018–2023) White Paper [Электронный ресурс]. — Режим доступа: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html> (дата обращения: 15.05.2026).
2. Whittaker S., Bergman O., Clough P. Easy on that trigger dad: a study of long term family photo retention // Personal and Ubiquitous Computing. — 2010. — Vol. 14, № 1. — P. 31–43.
3. Hu D. An introductory survey on attention mechanisms in NLP problems // Proceedings of SAI Intelligent Systems Conference (IntelliSys). — Springer, 2019. — P. 432–448.
4. Яндекс.Диск: возможности сервиса [Электронный ресурс]: официальный сайт. — Режим доступа: <https://disk.yandex.ru> (дата обращения: 15.05.2026).
5. Зайцева Л. В. Современные подходы к автоматической классификации изображений // Программные продукты и системы. — 2021. — № 2. — С. 245–253.
6. He K., Zhang X., Ren S., Sun J. Deep residual learning for image recognition // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2016. — P. 770–778.
7. Redmon J., Divvala S., Girshick R., Farhadi A. You Only Look Once: Unified, real-time object detection // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2016. — P. 779–788.
8. Radford A., Kim J. W., Hallacy C. et al. Learning transferable visual models from natural language supervision // Proceedings of the 38th International Conference on Machine Learning (ICML). — 2021. — Vol. 139. — P. 8748–8763.

9. Reimers N., Gurevych I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). — 2019. — P. 3982–3992.
10. Malkov Yu. A., Yashunin D. A. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2020. — Vol. 42, № 4. — P. 824–836.
11. Deng J., Guo J., Xue N., Zafeiriou S. ArcFace: Additive angular margin loss for deep face recognition // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2019. — P. 4690–4699.
12. McInnes L., Healy J., Astels S. HDBSCAN: Hierarchical density based clustering // Journal of Open Source Software. — 2017. — Vol. 2, № 11. — P. 205.
13. FastAPI: Modern, fast (high-performance), web framework for building APIs with Python [Электронный ресурс]: документация. — Режим доступа: <https://fastapi.tiangolo.com> (дата обращения: 15.05.2026).
14. MinIO: High performance object storage [Электронный ресурс]: документация. — Режим доступа: <https://min.io> (дата обращения: 15.05.2026).
15. Qdrant: Vector database and search engine [Электронный ресурс]: документация. — Режим доступа: <https://qdrant.tech> (дата обращения: 15.05.2026).
16. PostgreSQL: The world's most advanced open source relational database [Электронный ресурс]: документация. — Режим доступа: <https://www.postgresql.org> (дата обращения: 15.05.2026).

- 17.Nominatim: Geocoding with OpenStreetMap data [Электронный ресурс]: документация. — Режим доступа: <https://nominatim.org> (дата обращения: 15.05.2026).
- 18.InsightFace: 2D&3D deep face analysis library [Электронный ресурс]: репозиторий проекта. — Режим доступа: <https://github.com/deepinsight/insightface> (дата обращения: 15.05.2026).
- 19.Open CLIP: An implementation of CLIP [Электронный ресурс]: репозиторий проекта. — Режим доступа: https://github.com/mlfoundations/open_clip (дата обращения: 15.05.2026).
- 20.ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018. — 124 с.
- 21.ГОСТ Р 7.0.5–2008. Библиографическая ссылка. Общие требования и правила составления. — М.: Стандартинформ, 2008. — 23 с.
- 22.Воронцов К. В. Машинное обучение: курс лекций. — М.: МФТИ, 2022. — 312 с.
- 23.Николенко С. И., Кадури́н А. А., Архангельская Е. О. Глубокое обучение. — СПб.: Питер, 2018. — 480 с.
- 24.Гонсалес Р., Вудс Р. Цифровая обработка изображений / пер. с англ. — 3-е изд. — М.: Техносфера, 2012. — 1104 с.
- 25.Сайбель А. Г., Сухов И. М. Применение мультимодальных нейронных сетей для семантического поиска по медиаконтенту // Информационные технологии. — 2023. — Т. 29, № 4. — С. 195–203.
- 26.Семёнов А. Н., Кузнецов В. А. Распознавание лиц на основе глубоких сверточных нейронных сетей: обзор современных подходов // Известия высших учебных заведений. Приборостроение. — 2022. — Т. 65, № 6. — С. 421–435.

27. Гусев М. И. Векторные базы данных и их применение в задачах поиска по содержимому // Открытые системы. СУБД. — 2023. — № 1. — С. 32–37.
28. Кравченко В. Ю., Михайлов С. С. Архитектурные подходы к проектированию облачных хранилищ медиаконтента // Программная инженерия. — 2022. — Т. 13, № 8. — С. 376–384.
29. Орлов А. И., Левин Б. М. Современные технологии контейнеризации и оркестрации: Docker и Kubernetes в построении распределённых систем // Вестник СПбГУ. Серия Прикладная математика. Информатика. Процессы управления. — 2023. — Т. 19, № 2. — С. 178–192.
30. Малков Ю. А., Яшунин Д. А. Эффективный и устойчивый алгоритм приближённого поиска ближайших соседей в графах HNSW // Труды Института системного программирования РАН. — 2020. — Т. 32, № 4. — С. 89–108.
31. Малыгин Д. С. Асинхронные веб-приложения на Python: фреймворк FastAPI на практике // Системы и средства информатики. — 2023. — Т. 33, № 1. — С. 122–135.
32. Облако Mail.ru: облачное хранилище от VK [Электронный ресурс]: официальный сайт. — Режим доступа: <https://cloud.mail.ru> (дата обращения: 15.05.2026).
33. СберДиск: облачное хранилище данных [Электронный ресурс]: официальный сайт. — Режим доступа: <https://sberdisk.ru> (дата обращения: 15.05.2026).
34. Геокодер 2GIS: API обратного геокодирования [Электронный ресурс]: документация для разработчиков. — Режим доступа: <https://docs.2gis.ru/ru/api/search/geocoder/overview> (дата обращения: 18.05.2026).

35. Sentence-Transformers: фреймворк семантической близости текстов и изображений [Электронный ресурс]: документация. — Режим доступа: <https://www.sbert.net> (дата обращения: 18.05.2026).
36. Redis: документация ключ-значение хранилища [Электронный ресурс]. — Режим доступа: <https://redis.io/docs> (дата обращения: 15.05.2026).
37. ARQ: Job queues and RPC in Python with asyncio and Redis [Электронный ресурс]: документация. — Режим доступа: <https://arq-docs.helpmanual.io> (дата обращения: 15.05.2026).
38. Docker Compose: документация для разработчиков [Электронный ресурс]. — Режим доступа: <https://docs.docker.com/compose> (дата обращения: 15.05.2026).
39. HTMX: расширение HTML для современных пользовательских интерфейсов [Электронный ресурс]: документация. — Режим доступа: <https://htmx.org> (дата обращения: 19.05.2026).

